codebox/.style= colback=codedark!95!white, colframe=codeblue, colupper=white,
fontupper=, boxrule=0.5pt, arc=3pt,

Sentinel DV

Technical White Paper & Implementation Notes

DVCon USA / DAC 2026 — Companion Document

Open-Source Contribution

github.com/kiranreddi/sentinel-dv

2026

MIT License `pip install sentinel-dv`

Abstract

This document provides detailed technical notes to accompany the Sentinel DV conference paper [1]. It covers installation, configuration, tool API reference, adapter internals, intelligence algorithm details, and advanced deployment patterns. Intended for DV engineers evaluating or extending the framework.

Contents

1. Installation & Quick Start

1.1 Requirements

- Python 3.10, 3.11, or 3.12
- No EDA tool licenses required (parsers consume log/HTML artifacts only)
- Disk space: ∼50 MB base, plus artifact storage
- Network: none (all local; optional PyPI for install)

1.2 Install

```
pip install sentinel-dv
```

Or from source:

```
git clone https://github.com/kiranreddi/sentinel-dv\\
cd sentinel-dv\\
pip install -e ".[dev]"
```

1.3 Minimal Configuration

Create `sentinel_dv.yaml` in your project root:

Listing 1: Minimal sentinel_dv.yaml

```
# Sentinel DV configuration
artifact_roots:
  - /path/to/sim/logs          # directory containing *.log UVM logs
  - /path/to/coverage/report   # directory containing dashboard.html
  - /path/to/regression/xmls   # directory containing results.xml

db_path: ./sentinel_dv.db
```

1.4 Start the MCP Server

```
sentinel-dv serve --config sentinel\_dv.yaml
```

The server binds to `stdio` transport by default (required for Claude/VS Code Copilot integration). Add `-transport sse -port 8765` for network access.

1.5 Connect an AI Client

For Claude Desktop (`~/.config/claude/claude_desktop_config.json`):

Listing 2: Claude Desktop MCP config

```
{
  "mcpServers": {
    "sentinel-dv": {
      "command": "sentinel-dv",
      "args": ["serve", "--config", "sentinel_dv.yaml"]
    }
  }
}
```

For VS Code Copilot (`.vscode/mcp.json` in workspace):

Listing 3: VS Code Copilot MCP config

```
{
  "inputs": [],
  "servers": {
    "sentinel-dv": {
      "type": "stdio",
      "command": "sentinel-dv",
      "args": ["serve", "--config", "sentinel_dv.yaml"]
    }
  }
}
```

2. Configuration Reference

Listing 4: Full sentinel_dv.yaml reference

```
# === Artifact Discovery ===
artifact_roots:
  - /path/to/primary/artifacts
  - /path/to/coverage/reports

# === Database ===
db_path: ./sentinel_dv.db       # SQLite database path
auto_index: true                 # Re-index on server start

# === Coverage ===
coverage:
  html_glob_patterns:           # Glob patterns to find coverage HTML
    - "*/coverage/report/dashboard.html"    # VCS URG
    - "*/html_report/overalldu.js"          # Questa vcover
    - "*/coverage/index.html"              # Generic
  vcs_score_threshold: 75.0    # Warn if total < threshold
  questa_score_threshold: 75.0

# === Regression Health ===
health:
  pass_weight:      0.30
  coverage_weight:  0.35
  assertion_weight: 0.15
  flakiness_weight: 0.10
  cross_sim_weight: 0.10
  target_coverage:  75.0        # %  used in health computation

# === Path Redaction ===
redact_paths: true
redact_patterns:                # Additional regex patterns to redact
  - '/home/[^/]+'
  - '/workarea/[^/]+'

# === Simulator Support ===
simulators:
  - vcs
  - questa
  - xcelium
```

3. MCP Tool API Reference

All tools communicate via JSON-RPC 2.0. Inputs and outputs are Pydantic-validated. Below is a condensed reference; full schemas are in `sentinel_dv/tools/schemas.py`.

3.1 Run Tracking Tools

`runs.list`

Parameters: `page` (int, default 1), `page_size` (int, default 20), `simulator` (str, optional), `status` (str: pass/fail/all).
Returns: `runs` (list), `total` (int), `page_info` (object).

`runs.get`

Parameters: `run_id` (str, required).
Returns: Full run object under `run` key including test counts, timestamps, simulator, duration, log excerpts.

`runs.diff`

Parameters: `run_id_a`, `run_id_b` (str).
Returns: `added_failures`, `fixed_tests`, `coverage_delta`.

`runs.cross_sim`

Compares the same test suite across VCS, Questa, and Xcelium.
Returns: `simulators_compared`, `divergent_tests` list, `consistency_score` (0–100).

3.2 Coverage Intelligence Tools

`coverage.summary`

Parameters: `run_id` (str, optional — uses latest if omitted).
Returns: `summaries` list, each with `kind` (code/functional/assertion/plan), `metrics` list of {name, covered, total}.

Important: Returns `summaries` (list), not `summary` (singular).

`coverage.trend`

Parameters: `window` (int, default 10 runs), `metric` (str, default "total").
Returns: `data_points` list, `summary.direction` (improving/declining/stable).

Important: Direction is under `summary.direction`, not top-level.

coverage.gaps

Parameters: `threshold` (float, default 25.0 %).
Returns: `gaps` list with `scope`, `kind`, `covered_pct`.

coverage.advisor

Parameters: `gap_threshold` (float, default 25.0).
Returns: `advisories` list with `priority`, `sv_constraint`, `uvm_sequence`, `recommendation`.

3.3 Regression Health Tool

regression.health

No required parameters. Returns:

```
{
  "health_score": 77.5,          // 0-100 composite
  "band": "minor-issues",        // sign-off-ready | minor-issues |
                                 // coverage-gaps  | not-ready
  "grade": "B+",
  "components": {
    "pass_rate":    94.2,
    "coverage":     51.3,
    "assertions":   100.0,
    "flakiness":    98.1,
    "cross_sim":    87.5
  },
  "recommendations": [...]       // ranked list of action items
}
```

4. Adapter Layer Internals

4.1 Multi-Simulator UVM Log Parser

File: `sentinel_dv/adapters/uvm_log.py`

The parser uses five regex patterns, tried in priority order per log line:

1. **VCS native**: `UVM_ERROR file.sv(88) @ 500ns: comp [CAT] msg`
2. **Questa**: `# UVM_ERROR @ 500 ns: comp.path [CAT] msg`
3. **Xcelium**: `xcelium: UVM_ERROR @ 500ns: comp [CAT] msg`
4. **Generic UVM**: matches any remaining `UVM_(ERROR`FATAL)| line
5. **Count-summary skip**: `UVM_(ERROR`FATAL)*:*+| — **skipped**

False-positive elimination: Pattern 5 matches the summary lines that VCS appends to every log (e.g., `UVM_ERROR :  0`). Without this skip rule, 73% of reported failures are false positives.

4.2 VCS URG HTML Coverage Parser

File: `sentinel_dv/adapters/coverage_reports.py` — `_parse_urg_html()`

Algorithm:

1. Read `dashboard.html`; content-sniff for "Total Coverage Summary"
2. Locate the summary table via regex `<table[^>]*>.*?</table>`
3. For each `<tr>`, extract metric name and percentage value
4. Map vendor column names: SCOREtotal, LINEline, TOGGLEtoggle, etc.
5. Return `CoverageSummary` Pydantic object

Handled edge cases: numeric locale formatting (`87.30` vs `87,30`), N/A values, missing columns (old URG versions omit verification plan).

4.3 Questa vcover JS Coverage Parser

File: `sentinel_dv/adapters/coverage_reports.py` — `_parse_questa_overalldu()`

The `overalldu.js` file is generated as a single-line JavaScript assignment:

```
var overallDu = \{''items'': [\{''type'': ''stmt'', ''covered'': 1024,
    ''total'': 1200, ...\}]\};
```

Standard regex fails on deeply nested structures. The parser uses a **balanced-brace scanner**:

1. Locate the opening `{` after the assignment
2. Walk characters, tracking brace depth; collect substring until depth returns to 0
3. Parse the extracted substring as JSON (after minor JSJSON normalization)

Extracted fields: stmt, branch, toggle, fsm, assertion, functional, total. Each yields `hit` and `total` counts for percentage computation.

5. DV Intelligence Algorithms

5.1 Regression Health Score

The composite health score $H \in [0, 100]$ is:

$$H = w_P \cdot P + w_C \cdot C + w_A \cdot A + w_F \cdot (1 - F) + w_X \cdot X$$

where weights $(w_P, w_C, w_A, w_F, w_X) = (0.30, 0.35, 0.15, 0.10, 0.10)$ are configurable via `sentinel_dv.yaml`. Components:

- P: pass rate = passing tests / total tests (0–100)
- C: coverage score from `coverage.summary` total metric (0–100)
- A: assertion health = 100 if no assertion failures, else scaled (0–100)
- F: flakiness fraction = tests that diverge across simulator runs (0–1)
- X: cross-simulator consistency from `runs.cross_sim` (0–100)

Band thresholds are configurable. Default: 90/75/55 for sign-off/minor/gap bands.

5.2 Coverage Trend

For a window of N consecutive coverage reports ordered by timestamp, the trend direction is determined by a simple linear regression slope on the total coverage percentage series. Threshold: $|\text{slope}| < 0.5\%$/run is classified as `stable`; positive slope as `improving`; negative as `declining`.

5.3 Test Clustering

Failing test error messages are classified by string-matching heuristics:

- `timeout`: contains "timeout", "TIMEOUT", "time limit"
- `assertion`: UVM_FATAL or SVA `ASSERT_*` message
- `data_mismatch`: contains "MISMATCH", "Expected", "got"
- `uvm_phase`: contains "phase", "objection", "raise/drop"
- `unknown`: fallback category

Within each cluster, the representative failure is the most frequently occurring message pattern. Cluster size ranks recommendations.

5.4 Coverage Advisor Protocol Detection

The advisor detects the underlying protocol from signal names in coverage gaps:

Pattern in scope name	Protocol
`awaddr`, `araddr`, `awburst`	AXI4
`haddr`, `htrans`, `hburst`	AHB
`paddr`, `penable`, `psel`	APB
`txreq`, `rxdat`, `txrsp`	CHI
`tl_a_`, `tl_d_`	TileLink
`pcie_`, `gen3_`	PCIe

6. AXI4 UVM Demo

The repository includes a complete AXI4-Lite verification demo in `demo/axi4_uvm/`.

6.1 Demo Structure

```
demo/axi4\_uvm/\\
  config.yaml            \# Sentinel DV config for the demo\\
  vcs/                   \# VCS simulator artifacts\\
    results.xml          \# JUnit-format test results\\
    assertions/          \# Assertion JSON files\\
    coverage/            \# dashboard.html (VCS URG)\\
    waveforms/           \# .wave.json waveform summaries\\
  questa/                \# Questa simulator artifacts (same layout)\\
```

```
  xcelium/                \# Xcelium simulator artifacts (same layout)\\
  README.md               \# How to run and use
```

6.2 Test Cases

Four test cases cover the AXI4-Lite verification plan:

1. `axi4_bk2bk_test`: Back-to-back write/read transactions
2. `axi4_err_resp_test`: Error response checking (SLVERR/DECERR)
3. `axi4_boundary_test`: 4KB boundary alignment
4. `axi4_stress_test`: Random burst stress (reveals cross-sim divergence)

6.3 Running the Demo

```
cd sentinel-dv\\
sentinel-dv index --config demo/axi4\_uvm/config.yaml\\
\# Then open Claude Desktop and ask:\\
\# "What's the health of the AXI4 demo regression?"
```

Expected output: health score ∼72, band `coverage-gaps`, 3 divergent tests between Questa and VCS/Xcelium for `axi4_stress_test`.

7. Extending Sentinel DV

7.1 Adding a New Adapter

1. Create `sentinel_dv/adapters/my_format.py`
2. Implement `parse(path: str) -> List[Artifact]` function
3. Register in `sentinel_dv/adapters/__init__.py`
4. Add glob patterns to `ARTIFACT_GLOBS` in `indexer.py`
5. Write unit tests in `tests/unit/test_my_format.py`

7.2 Adding a New MCP Tool

1. Add tool function to appropriate module in `sentinel_dv/tools/`
2. Decorate with `@mcp.tool()` from `fastmcp`
3. Define Pydantic input/output schemas in `tools/schemas.py`
4. Register in `sentinel_dv/server.py`
5. Add integration test in `tests/integration/`

8. CI/CD Integration

8.1 GitHub Actions

Listing 5: .github/workflows/sentinel-dv.yml

```yaml
name: Sentinel DV Analysis
on: [push]

jobs:
  dv-analysis:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install Sentinel DV
        run: pip install sentinel-dv
      - name: Download sim artifacts
        run: |
          # Copy/mount your simulation artifacts
          cp -r $ARTIFACT_DIR ./sim_artifacts
      - name: Run Sentinel DV health check
        run: |
          sentinel-dv index --config .sentinel_dv.yaml
          sentinel-dv check-health --min-score 70
```

8.2 Jenkins Pipeline

Listing 6: Jenkinsfile snippet

```
stage('DV Quality Gate') {
    steps {
        sh 'pip install sentinel-dv'
        sh 'sentinel-dv index --config sentinel_dv.yaml'
        sh 'sentinel-dv check-health --min-score ${params.MIN_HEALTH
            :-70}'
    }
}
```

9. Troubleshooting

9.1 Common Issues

Coverage not found: Ensure the HTML files exist at the paths in `artifact_roots`. Run `sentinel-dv list-artifacts` to see what was discovered.

0 runs indexed: Check that `artifact_roots` contains directories with `*.log` or `results.xml` files. VCS logs must contain `UVM_ERROR` or `Errors:` lines to be detected as simulation logs.

Health score returns 0: Run `regression.health` with `-debug` flag or check that at least one run is indexed with `runs.list`.

coverage.summary returns empty summaries: The HTML parser requires the vendor-specific report format. Ensure you are using `urg -report` (VCS) or `vcover report -html` (Questa).

10. Future Roadmap

- **v2.2.0**: Direct binary database parsing (UCDB via `questa-vpm`, VDB via `vcs-dv`)
- **v2.3.0**: FSDB/MXDB waveform signals and transitions

- **v3.0.0**: Formal property synthesis from coverage gap analysis
- **v3.1.0**: Cloud MCP endpoint + Docker image for CI/CD
- **v3.2.0**: Test generation from failure clusters using LLM fine-tuning
- **v4.0.0**: Testplan tracking with coverage closure metrics

References

References

[1] Sentinel DV Conference Paper, DVCon USA 2026. `conference/paper/sentinel_dv_dvcon2026.pdf`

[2] Anthropic, "Model Context Protocol," https://modelcontextprotocol.io, 2024.

[3] J. Lowin, "FastMCP," https://github.com/jlowin/fastmcp, 2024.